S004 Aaryan Raorane

Python for Data Science: Experiments 6 to 14

Experiment 6

In [1]:
```
import pandas as pd
import numpy as np
```

```
C:\Users\djsce.student\AppData\Local\Continuum\anaconda3\lib\site-packages\pandas\compat\_optional.py:138: UserW
arning: Pandas requires version '2.7.0' or newer of 'numexpr' (version '2.6.8' currently installed).
  warnings.warn(msg, UserWarning)
```

In [2]:
```
#panda series 10 random integer find mean , access the first 5 elements through slicing
np.random.seed(0)
sr=pd.Series(np.random.randint(1,101,size=10))
print(sr)
print(np.mean(sr))
sr_slice=sr.loc[0:4]
print("\nSliced_Rows :-")
print(sr_slice)
```

```
0    45
1    48
2    65
3    68
4    68
5    10
6    84
7    22
8    37
9    88
dtype: int32
53.5

Sliced_Rows :-
0    45
1    48
2    65
3    68
4    68
dtype: int32
```

In [3]:
```
#create data frame from dictionary
data = {'Name' : ['John','Emma','Michael','Sophia','Aadi'],
        'Age' : [25,30,35,28,69],
        'City' : ['New York','Paris','London','Sydney','Kalyan']}
df = pd.DataFrame(data)
print(df)
```

```
      Name  Age      City
0     John   25  New York
1     Emma   30     Paris
2  Michael   35    London
3   Sophia   28    Sydney
4     Aadi   69    Kalyan
```

In [4]:
```
#create datafram from the following series
s1 = pd.Series(['Python','ML','NLP','DL'])
s2 = pd.Series(['9','8','7','9'])
s3 = pd.Series(['7','8','9','9'])
df=pd.concat([s1,s2,s3],axis=1)
print(df)
```

```
        0  1  2
0  Python  9  7
1      ML  8  8
2     NLP  7  9
3      DL  9  9
```

In [5]:
```
#create data frame from dictionary
data = {'Name' : ['John','Emma','Michael','Sophia'],
        'Age' : [25,30,35,28],
        'City' : ['New York','Paris','London','Sydney']}
df = pd.DataFrame(data)
print(df)
```

```
print()
df['Scores']=pd.Series([85,90,78,92])
print(df)
print()
print(df.iloc[0])
print()
age25 = df[df["Age"]==25]
print(age25)
print()
df.drop('City',axis=1,inplace=True)
print(df)
print()
agem30 =df[df["Age"]>30]
print(agem30)
print()
df.rename(columns={'Name':'FullName'},inplace=True)
print(df)
```

```
      Name  Age      City
0     John   25  New York
1     Emma   30     Paris
2  Michael   35    London
3   Sophia   28    Sydney

      Name  Age      City  Scores
0     John   25  New York      85
1     Emma   30     Paris      90
2  Michael   35    London      78
3   Sophia   28    Sydney      92

Name          John
Age             25
City      New York
Scores          85
Name: 0, dtype: object

   Name  Age      City  Scores
0  John   25  New York      85

      Name  Age  Scores
0     John   25      85
1     Emma   30      90
2  Michael   35      78
3   Sophia   28      92

      Name  Age  Scores
2  Michael   35      78

  FullName  Age  Scores
0     John   25      85
1     Emma   30      90
2  Michael   35      78
3   Sophia   28      92
```

Experiment 7

In [6]:
```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
```

In [7]:
```
titanic = pd.read_csv("titanic.csv")
```

In [8]:
```
titanic.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 891 entries, 0 to 890
Data columns (total 12 columns):
 #   Column       Non-Null Count  Dtype  
---  ------       --------------  -----  
 0   PassengerId  891 non-null    int64  
 1   Survived     891 non-null    int64  
 2   Pclass       891 non-null    int64  
 3   Name         891 non-null    object 
 4   Sex          891 non-null    object 
 5   Age          714 non-null    float64
 6   SibSp        891 non-null    int64  
 7   Parch        891 non-null    int64  
 8   Ticket       891 non-null    object 
 9   Fare         891 non-null    float64
 10  Cabin        204 non-null    object 
 11  Embarked     889 non-null    object 
dtypes: float64(2), int64(5), object(5)
memory usage: 83.7+ KB
```

In [9]:
```
titanic['Age'] = titanic['Age'].fillna(titanic['Age'].median())
```

In [10]:
```
titanic.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 891 entries, 0 to 890
Data columns (total 12 columns):
 #   Column       Non-Null Count  Dtype  
---  ------       --------------  -----  
 0   PassengerId  891 non-null    int64  
 1   Survived     891 non-null    int64  
 2   Pclass       891 non-null    int64  
 3   Name         891 non-null    object 
 4   Sex          891 non-null    object 
 5   Age          891 non-null    float64
 6   SibSp        891 non-null    int64  
 7   Parch        891 non-null    int64  
 8   Ticket       891 non-null    object 
 9   Fare         891 non-null    float64
 10  Cabin        204 non-null    object 
 11  Embarked     889 non-null    object 
dtypes: float64(2), int64(5), object(5)
memory usage: 83.7+ KB
```

In [11]:
```
from scipy import stats

titanic['z_score'] = stats.zscore(titanic['Fare'])
outliers = titanic[titanic['z_score'].abs() > 3]
outliers
```

Out[11]:

	PassengerId	Survived	Pclass	Name	Sex	Age	SibSp	Parch	Ticket	Fare	Cabin	Embarked
27	28	0	1	Fortune, Mr. Charles Alexander	male	19.0	3	2	19950	263.0000	C23 C25 C27	S
88	89	1	1	Fortune, Miss. Mabel Helen	female	23.0	3	2	19950	263.0000	C23 C25 C27	S
118	119	0	1	Baxter, Mr. Quigg Edmond	male	24.0	0	1	PC 17558	247.5208	B58 B60	C
258	259	1	1	Ward, Miss. Anna	female	35.0	0	0	PC 17755	512.3292	NaN	C
299	300	1	1	Baxter, Mrs. James (Helene DeLaudeniere Chaput)	female	50.0	0	1	PC 17558	247.5208	B58 B60	C
311	312	1	1	Ryerson, Miss. Emily Borie	female	18.0	2	2	PC 17608	262.3750	B57 B59 B63 B66	C
341	342	1	1	Fortune, Miss. Alice Elizabeth	female	24.0	3	2	19950	263.0000	C23 C25 C27	S
377	378	0	1	Widener, Mr. Harry Elkins	male	27.0	0	2	113503	211.5000	C82	C
380	381	1	1	Bidois, Miss. Rosalie	female	42.0	0	0	PC 17757	227.5250	NaN	C
438	439	0	1	Fortune, Mr. Mark	male	64.0	1	4	19950	263.0000	C23 C25 C27	S
527	528	0	1	Farthing, Mr. John	male	28.0	0	0	PC 17483	221.7792	C95	S
557	558	0	1	Robbins, Mr. Victor	male	28.0	0	0	PC 17757	227.5250	NaN	C
679	680	1	1	Cardeza, Mr. Thomas Drake Martinez	male	36.0	0	1	PC 17755	512.3292	B51 B53 B55	C
689	690	1	1	Madill, Miss. Georgette Alexandra	female	15.0	0	1	24160	211.3375	B5	S
700	701	1	1	Astor, Mrs. John Jacob (Madeleine Talmadge Force)	female	18.0	1	0	PC 17757	227.5250	C62 C64	C
716	717	1	1	Endres, Miss. Caroline Louise	female	38.0	0	0	PC 17757	227.5250	C45	C
730	731	1	1	Allen, Miss. Elisabeth Walton	female	29.0	0	0	24160	211.3375	B5	S
737	738	1	1	Lesurer, Mr. Gustave J	male	35.0	0	0	PC 17755	512.3292	B101	C
742	743	1	1	Ryerson, Miss. Susan Parker "Suzette"	female	21.0	2	2	PC 17608	262.3750	B57 B59 B63 B66	C
779	780	1	1	Robert, Mrs. Edward Scott (Elisabeth Walton Mc...	female	43.0	0	1	24160	211.3375	B3	S

In [12]:

```
iris = pd.read_csv("Iris.csv")
```

In [13]:
```
iris.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 150 entries, 0 to 149
Data columns (total 6 columns):
 #   Column         Non-Null Count  Dtype  
---  ------         --------------  -----  
 0   Id             150 non-null    int64  
 1   SepalLengthCm  150 non-null    float64
 2   SepalWidthCm   150 non-null    float64
 3   PetalLengthCm  150 non-null    float64
 4   PetalWidthCm   150 non-null    float64
 5   Species        150 non-null    object 
dtypes: float64(4), int64(1), object(1)
memory usage: 7.2+ KB
```

In [14]:
```
q1 = iris['SepalLengthCm'].quantile(0.25)
q3 = iris['SepalLengthCm'].quantile(0.75)
iqr = q3 - q1
lower_bound = q1 - 1.5 * iqr
upper_bound = q3 + 1.5 * iqr
outliers = iris[(iris['SepalLengthCm'] < lower_bound) | (iris['SepalLengthCm'] > upper_bound)]
print(outliers)
```

```
Empty DataFrame
Columns: [Id, SepalLengthCm, SepalWidthCm, PetalLengthCm, PetalWidthCm, Species]
Index: []
```

In [15]:
```
iris['Z_Score'] = stats.zscore(iris['PetalWidthCm'])
outliers = iris[(iris['Z_Score'] < -3) | (iris['Z_Score'] > 3)]
print(outliers)
```

```
Empty DataFrame
Columns: [Id, SepalLengthCm, SepalWidthCm, PetalLengthCm, PetalWidthCm, Species, Z_Score]
Index: []
```

In [16]:
```
from sklearn.datasets import load_boston
boston = load_boston()
df = pd.DataFrame(boston.data, columns=boston.feature_names)
df['MEDV'] = boston.target
print(df.info())
print(df.head())
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 506 entries, 0 to 505
Data columns (total 14 columns):
 #   Column   Non-Null Count  Dtype  
---  ------   --------------  -----  
 0   CRIM     506 non-null    float64
 1   ZN       506 non-null    float64
 2   INDUS    506 non-null    float64
 3   CHAS     506 non-null    float64
 4   NOX      506 non-null    float64
 5   RM       506 non-null    float64
 6   AGE      506 non-null    float64
 7   DIS      506 non-null    float64
 8   RAD      506 non-null    float64
 9   TAX      506 non-null    float64
 10  PTRATIO  506 non-null    float64
 11  B        506 non-null    float64
 12  LSTAT    506 non-null    float64
 13  MEDV     506 non-null    float64
dtypes: float64(14)
memory usage: 55.5 KB
None
      CRIM    ZN  INDUS  CHAS    NOX     RM   AGE     DIS  RAD    TAX  \
0  0.00632  18.0   2.31   0.0  0.538  6.575  65.2  4.0900  1.0  296.0   
1  0.02731   0.0   7.07   0.0  0.469  6.421  78.9  4.9671  2.0  242.0   
2  0.02729   0.0   7.07   0.0  0.469  7.185  61.1  4.9671  2.0  242.0   
3  0.03237   0.0   2.18   0.0  0.458  6.998  45.8  6.0622  3.0  222.0   
4  0.06905   0.0   2.18   0.0  0.458  7.147  54.2  6.0622  3.0  222.0   

   PTRATIO       B  LSTAT  MEDV  
0     15.3  396.90   4.98  24.0  
1     17.8  396.90   9.14  21.6  
2     17.8  392.83   4.03  34.7  
3     18.7  394.63   2.94  33.4  
4     18.7  396.90   5.33  36.2  
```

In [17]:
```
df = df.dropna(subset=['RM'])
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
Int64Index: 506 entries, 0 to 505
Data columns (total 14 columns):
 #   Column   Non-Null Count  Dtype  
---  ------   --------------  -----  
 0   CRIM     506 non-null    float64
 1   ZN       506 non-null    float64
 2   INDUS    506 non-null    float64
 3   CHAS     506 non-null    float64
 4   NOX      506 non-null    float64
 5   RM       506 non-null    float64
 6   AGE      506 non-null    float64
 7   DIS      506 non-null    float64
 8   RAD      506 non-null    float64
 9   TAX      506 non-null    float64
 10  PTRATIO  506 non-null    float64
 11  B        506 non-null    float64
 12  LSTAT    506 non-null    float64
 13  MEDV     506 non-null    float64
dtypes: float64(14)
memory usage: 59.3 KB
```

In [18]:
```
q1 = df['AGE'].quantile(0.25)
q3 = df['AGE'].quantile(0.75)
iqr = q3 - q1
lower_bound = q1 - 1.5 * iqr
upper_bound = q3 + 1.5 * iqr
outliers = df[(df['AGE'] < lower_bound) | (df['AGE'] > upper_bound)]
print(outliers['AGE'])
```

```
Series([], Name: AGE, dtype: float64)
```

In [19]:
```
wine = pd.read_csv("winequality-red.csv")
wine.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 1599 entries, 0 to 1598
Data columns (total 12 columns):
 #   Column                Non-Null Count  Dtype  
---  ------                --------------  -----  
 0   fixed acidity         1599 non-null   float64
 1   volatile acidity      1599 non-null   float64
 2   citric acid           1599 non-null   float64
 3   residual sugar        1599 non-null   float64
 4   chlorides             1599 non-null   float64
 5   free sulfur dioxide   1599 non-null   float64
 6   total sulfur dioxide  1599 non-null   float64
 7   density               1599 non-null   float64
 8   pH                    1599 non-null   float64
 9   sulphates             1599 non-null   float64
 10  alcohol               1599 non-null   float64
 11  quality               1599 non-null   int64  
dtypes: float64(11), int64(1)
memory usage: 150.0 KB
```

In [20]:
```
wine['total sulfur dioxide'] = wine['total sulfur dioxide'].fillna(wine['total sulfur dioxide'].mean())
```

In [21]:
```
wine['fixed acidity'] = wine['fixed acidity'].fillna(0)
print(wine)
```

```
      fixed acidity  volatile acidity  citric acid  residual sugar  chlorides  \
0               7.4             0.700         0.00             1.9      0.076
1               7.8             0.880         0.00             2.6      0.098
2               7.8             0.760         0.04             2.3      0.092
3              11.2             0.280         0.56             1.9      0.075
4               7.4             0.700         0.00             1.9      0.076
...             ...               ...          ...             ...        ...
1594            6.2             0.600         0.08             2.0      0.090
1595            5.9             0.550         0.10             2.2      0.062
1596            6.3             0.510         0.13             2.3      0.076
1597            5.9             0.645         0.12             2.0      0.075
1598            6.0             0.310         0.47             3.6      0.067

      free sulfur dioxide  total sulfur dioxide  density    pH  sulphates  \
0                    11.0                  34.0  0.99780  3.51       0.56
1                    25.0                  67.0  0.99680  3.20       0.68
2                    15.0                  54.0  0.99700  3.26       0.65
3                    17.0                  60.0  0.99800  3.16       0.58
4                    11.0                  34.0  0.99780  3.51       0.56
...                   ...                   ...      ...   ...        ...
1594                 32.0                  44.0  0.99490  3.45       0.58
1595                 39.0                  51.0  0.99512  3.52       0.76
1596                 29.0                  40.0  0.99574  3.42       0.75
1597                 32.0                  44.0  0.99547  3.57       0.71
1598                 18.0                  42.0  0.99549  3.39       0.66

      alcohol  quality
0         9.4        5
1         9.8        5
2         9.8        5
3         9.8        6
4         9.4        5
...       ...      ...
1594     10.5        5
1595     11.2        6
1596     11.0        6
1597     10.2        5
1598     11.0        6

[1599 rows x 12 columns]
```

Experiments 8 and 9

In [22]:
```python
# Q1: Leap year check
def is_leap(y):
    return (y % 4 == 0 and y % 100 != 0) or (y % 400 == 0)

for y in [2000, 1900, 2024, 2023]:
    print(y, "Leap year" if is_leap(y) else "Not a leap year")
```

```
2000 Leap year
1900 Not a leap year
2024 Leap year
2023 Not a leap year
```

In [23]:
```python
# Q2: Extract even numbers from a list
def ExtractEven(lst):
    return [x for x in lst if x % 2 == 0]

print(ExtractEven([1, 2, 3, 4, 5, 6]))
```

```
[2, 4, 6]
```

In [24]:
```python
# Q3: Add, sub, mul, div of two numbers
def calc(a, b):
    return a + b, a - b, a * b, (a / b if b != 0 else "Division by zero")

print(calc(20, 5))
```

```
(25, 15, 100, 4.0)
```

In [25]:
```python
# Q4: Area of circle (1 argument) or triangle (2 arguments)
def area(a, b=None):
    if b is None:
        return 3.14159 * a * a
    return 0.5 * a * b

print("Circle:", area(5))
print("Triangle:", area(10, 4))
```

```
Circle: 78.53975
Triangle: 20.0
```

In [26]:
```python
# Q5: Largest number using variable length arguments
def largest(*nums):
    return max(nums)

print(largest(4, 9))
print(largest(4, 9, 15))
print(largest(4, 9, 15, 2, 30))
```

```
9
15
30
```

In [27]:
```python
# Q6: Demonstrate **kwargs
def show_info(**kwargs):
    for k, v in kwargs.items():
        print(k, ":", v)

show_info(name="Aaryan", branch="AI and DS", year=2)
```

```
name : Aaryan
branch : AI and DS
year : 2
```

In [28]:
```python
# Q7: Fibonacci series up to n terms using recursion
def fib(n):
    if n <= 1:
        return n
    return fib(n - 1) + fib(n - 2)

n = 10
print([fib(i) for i in range(n)])
```

```
[0, 1, 1, 2, 3, 5, 8, 13, 21, 34]
```

In [29]:
```python
# Q8: Maximum value in a list using recursion
def find_max(lst):
    if len(lst) == 1:
        return lst[0]
    m = find_max(lst[1:])
    return lst[0] if lst[0] > m else m

print(find_max([3, 8, 1, 25, 7]))
```

```
25
```

In [30]:
```python
# Q9: Length of a string using recursion
def string_length(s):
    if s == "":
        return 0
    return 1 + string_length(s[1:])

print(string_length("Python"))
```

```
6
```

In [31]:
```python
# Q10: Power using recursion
def power(base, exp):
    if exp == 0:
        return 1
    return base * power(base, exp - 1)

print(power(2, 5))
```

```
32
```

In [32]:
```python
# Q11: Print star pattern in n rows using recursion
def print_pattern(n):
    if n == 0:
        return
    print_pattern(n - 1)
    print("*" * n)

print_pattern(5)
```

```
*
**
***
****
*****
```

In [33]:
```python
# Q12: Factorial using recursion
def factorial(n):
    if n <= 1:
```

```
        return 1
    return n * factorial(n - 1)

print(factorial(5))
```

```
120
```

In [34]:
```
# Q13: Merge two dictionaries, overlapping keys stored as a list
def merge_dicts(d1, d2):
    result = dict(d1)
    for k, v in d2.items():
        if k in result:
            result[k] = [result[k], v]
        else:
            result[k] = v
    return result

print(merge_dicts({"a": 1, "b": 2}, {"b": 3, "c": 4}))
```

```
{'a': 1, 'b': [2, 3], 'c': 4}
```

In [35]:
```
# Q14: Squares of first 10 Fibonacci numbers using map
fibs = [fib(i) for i in range(10)]
print(list(map(lambda x: x * x, fibs)))
```

```
[0, 1, 1, 4, 9, 25, 64, 169, 441, 1156]
```

In [36]:
```
# Q15: Lambda to add 26, and lambda to multiply x and y
add26 = lambda x: x + 26
mul = lambda x, y: x * y
print(add26(10))
print(mul(6, 7))
```

```
36
42
```

In [37]:
```
# Q16: Numbers divisible by 9 using lambda
nums = [9, 12, 18, 25, 27, 40, 45, 50]
print(list(filter(lambda x: x % 9 == 0, nums)))
```

```
[9, 18, 27, 45]
```

In [38]:
```
# Q17 i: Filter out inactive users (no tweets)
users = [
    {"username": 'seema', "tweets": ["i love chocolate", "i am a student"]},
    {"username": 'arush', "tweets": []},
    {"username": 'kumal', "tweets": ["India", "Python"]},
    {"username": 'sam', "tweets": []},
    {"username": 'lokesh', "tweets": ["i am good"]},
]

inactive = list(filter(lambda u: not u["tweets"], users))
print(inactive)
```

```
[{'username': 'arush', 'tweets': []}, {'username': 'sam', 'tweets': []}]
```

In [39]:
```
# Q17 ii: Inactive users, only username in uppercase
print(list(map(lambda u: u["username"].upper(), filter(lambda u: not u["tweets"], users))))
```

```
['ARUSH', 'SAM']
```

In [40]:
```
# Q17 iii: "your name is" + name, for names with length 4 or more
names = ['Rohan', 'Rhea', 'bob', 'to']
print(list(map(lambda n: "your name is " + n, filter(lambda n: len(n) >= 4, names))))
```

```
['your name is Rohan', 'your name is Rhea']
```

Experiment 10

In [41]:
```
"""
1. Implement a program to handle arithmetic exception
"""
try:
    num1 = int(input("Enter first number: "))
    num2 = int(input("Enter second number: "))
    result = num1 / num2
    print("Result is", result)
except ZeroDivisionError:
    print("Error: You cannot divide by zero.")
```

```
Enter first number: 0
Enter second number: 0
Error: You cannot divide by zero.
```

In [42]:
```
"""
2. Write a program to handle following exceptions
a. Type Error
b. Value Error
c. Make use of else with exception and check its effect
"""
try:
    data = input("Enter a number: ")
    number = int(data)
    result = data + number
except ValueError:
    print("ValueError: You did not enter a valid integer.")
except TypeError:
    print("TypeError: Cannot add a string to an integer.")
else:
    print("Success: No errors occurred. Result is", result)
```

```
Enter a number: 0
TypeError: Cannot add a string to an integer.
```

In [43]:
```
"""
3. Write a program to write 1-20 numbers in a file & display same to the user.
"""
file = open("numbers.txt", "w")
for i in range(1, 21):
    file.write(str(i) + "\n")
file.close()
print("Numbers written to file. Reading now:")
file = open("numbers.txt", "r")
content = file.read()
print(content)
file.close()
```

```
Numbers written to file. Reading now:
1
2
3
4
5
6
7
8
9
10
11
12
13
14
15
16
17
18
19
20
```

In [44]:
```
"""
4. Write a program to copy contents (your details) of one file to another file and display contents of both the
"""
file = open("details.txt", "w")
file.write("Name: Aarav Sharma\n")
file.write("Course: B.Tech AI & DS\n")
file.write("College: Mumbai University\n")
file.close()

source = open("details.txt", "r")
content = source.read()
source.close()

destination = open("copy_details.txt", "w")
destination.write(content)
destination.close()

print("Contents of details.txt:")
source = open("details.txt", "r")
print(source.read())
source.close()

print("Contents of copy_details.txt:")
destination = open("copy_details.txt", "r")
print(destination.read())
destination.close()
```

```
Contents of details.txt:
Name: Aarav Sharma
Course: B.Tech AI & DS
College: Mumbai University

Contents of copy_details.txt:
Name: Aarav Sharma
Course: B.Tech AI & DS
College: Mumbai University
```

In [45]:
```
"""
5. Write a python script to continue writing to a file until user enters choice as y and then on user input n s
"""
file = open("user_text.txt", "w")

choice = 'y'
while choice == 'y':
    text = input("Enter text to write: ")
    file.write(text + "\n")
    choice = input("Do you want to continue? (y/n): ")

file.close()

file = open("user_text.txt", "r")
words = file.read().split()
file.close()

count = 0
for word in words:
    if word.endswith('e'):
        count += 1

print("Number of words ending with 'e':", count)
```

```
Number of words ending with 'e': 1
```

In [46]:
```
"""
6. Write a python script to count the number of lines from a text file which is not starting with an alphabet "
"""
file = open("sample.txt", "w")
file.write("This is line one\n")
file.write("Another line here\n")
file.write("Testing the script\n")
file.write("Python is fun\n")
file.close()

file = open("sample.txt", "r")
lines = file.readlines()
file.close()

count = 0
for line in lines:
    if not line.startswith('T'):
        count += 1

print("Number of lines not starting with 'T':", count)
```

```
Number of lines not starting with 'T': 2
```

In [47]:
```
"""
7. Write a python script to delete the specified line giving the line number from the text file.
"""
file = open("lines.txt", "w")
file.write("Line 1\n")
file.write("Line 2\n")
file.write("Line 3\n")
file.write("Line 4\n")
file.write("Line 5\n")
file.close()

line_no = int(input("Enter line number to delete: "))

file = open("lines.txt", "r")
lines = file.readlines()
file.close()

del lines[line_no - 1]

file = open("lines.txt", "w")
file.writelines(lines)
file.close()

print("Line", line_no, "deleted successfully.")
```

```
file = open("lines.txt", "r")
print(file.read())
file.close()
```

```
Line 2 deleted successfully.
Line 1
Line 3
Line 4
Line 5
```

In [48]:
```
"""
8. Program to set file offsets with seek( ) and tell( ) functions in python
"""
file = open("offset_demo.txt", "w")
file.write("Hello, this is a file offset demonstration.")
file.close()

file = open("offset_demo.txt", "r")

print("Current position:", file.tell())

file.seek(7)
print("After seek(7), position:", file.tell())
print("Reading from position 7:", file.read(10))

file.seek(0)
print("After seek(0), position:", file.tell())
print("Reading from start:", file.read(5))

file.close()
```

```
Current position: 0
After seek(7), position: 7
Reading from position 7: this is a
After seek(0), position: 0
Reading from start: Hello
```

Experiment 11

Practical 11: Object Oriented Programming

Aim: To implement Object Oriented Programming concepts in Python (class, object, self, `__init__`, instance variables, methods, inheritance, overriding and overloading).

Q1. Create a Vehicle class without any variables and methods

In [49]:
```
class Vehicle:
    pass          # empty class body

v = Vehicle()     # create an object of the class
print("Vehicle object created successfully.")
print("Type of object:", type(v))
```

```
Vehicle object created successfully.
Type of object: <class '__main__.Vehicle'>
```

Q2. Create a Vehicle class with max_speed and mileage instance attributes

In [50]:
```
class Vehicle:
    def __init__(self, max_speed, mileage):
        self.max_speed = max_speed      # instance variable
        self.mileage = mileage          # instance variable

    def display(self):
        print(f"Max speed : {self.max_speed} km/h")
        print(f"Mileage   : {self.mileage} km/l")

car = Vehicle(180, 18)
bike = Vehicle(120, 45)

print("Car details")
car.display()
```

```
print()
print("Bike details")
bike.display()
```

```
Car details
Max speed : 180 km/h
Mileage   : 18 km/l

Bike details
Max speed : 120 km/h
Mileage   : 45 km/l
```

Q3. Create a child class Bus that will inherit all of the variables and methods of the Vehicle class

In [51]:
```
class Vehicle:
    def __init__(self, name, max_speed, mileage):
        self.name = name
        self.max_speed = max_speed
        self.mileage = mileage

class Bus(Vehicle):
    pass          # inherits everything from Vehicle

print("Class Bus created. Parent class of Bus:", Bus.__bases__[0].__name__)
```

```
Class Bus created. Parent class of Bus: Vehicle
```

Q4. Create a Bus object that will inherit all of the variables and methods of the parent Vehicle class and display it

Expected Output: `Vehicle Name: School Volvo Speed: 180 Mileage: 12`

In [52]:
```
class Vehicle:
    def __init__(self, name, max_speed, mileage):
        self.name = name
        self.max_speed = max_speed
        self.mileage = mileage

class Bus(Vehicle):
    pass

School_bus = Bus("School Volvo", 180, 12)
print("Vehicle Name:", School_bus.name, "Speed:", School_bus.max_speed, "Mileage:", School_bus.mileage)
```

```
Vehicle Name: School Volvo Speed: 180 Mileage: 12
```

Q5. Create a Bus class that inherits from the Vehicle class. Give the capacity argument of Bus.seating_capacity() a default value of 50

Expected Output: `The seating capacity of a bus is 50 passengers`

In [53]:
```
class Vehicle:
    def __init__(self, name, max_speed, mileage):
        self.name = name
        self.max_speed = max_speed
        self.mileage = mileage

    def seating_capacity(self, capacity):
        return f"The seating capacity of a {self.name} is {capacity} passengers"

class Bus(Vehicle):
    # override the parent method and give capacity a default value of 50
    def seating_capacity(self, capacity=50):
        return super().seating_capacity(capacity)

School_bus = Bus("bus", 180, 12)
print(School_bus.seating_capacity())
```

```
The seating capacity of a bus is 50 passengers
```

Q6. Class Inheritance

The default fare charge of any vehicle is **seating capacity * 100**. If the Vehicle is a **Bus** instance, add an extra 10% of the total fare as a maintenance charge, so the final amount = total fare + 10% of the total fare.

Expected Output: `Total Bus fare is: 5500.0`

In [54]:
```
class Vehicle:
    def __init__(self, name, mileage, capacity):
        self.name = name
        self.mileage = mileage
        self.capacity = capacity

    def fare(self):
        return self.capacity * 100

class Bus(Vehicle):
    def fare(self):
        amount = super().fare()
        amount += amount * 10 / 100      # extra 10% maintenance charge
        return amount

School_bus = Bus("School Volvo", 12, 50)
print("Total Bus fare is:", School_bus.fare())
```

```
Total Bus fare is: 5500.0
```

Q7. Check type of an object

Write a program to determine which class a given Bus object belongs to.

In [55]:
```
print(type(School_bus))
print("Class name:", type(School_bus).__name__)
```

```
<class '__main__.Bus'>
Class name: Bus
```

Q8. Determine if School_bus is also an instance of the Vehicle class

In [56]:
```
print(isinstance(School_bus, Vehicle))
```

```
True
```

Q9. Determine if School_bus is a sub class of the Vehicle class

In [57]:
```
print(issubclass(type(School_bus), Vehicle))
print(issubclass(Bus, Vehicle))
```

```
True
True
```

Q10. Create a child class for Bus class named Minibus inheriting the Bus class with a price attribute and a Print Price method (Multilevel Inheritance)

In [58]:
```
class Vehicle:
    def __init__(self, name, mileage, capacity):
        self.name = name
        self.mileage = mileage
        self.capacity = capacity

    def fare(self):
        return self.capacity * 100

class Bus(Vehicle):
    pass

class Minibus(Bus):
    def __init__(self, name, mileage, capacity, price):
        super().__init__(name, mileage, capacity)   # access the parent class from inside the child
        self.price = price

    def print_price(self):
        print("Price of", self.name, "is:", self.price)

mini = Minibus("Mini Volvo", 15, 20, 2500000)
mini.print_price()
print("Total Bus fare is:", mini.fare())
print("MRO:", [c.__name__ for c in Minibus.__mro__])
```

```
Price of Mini Volvo is: 2500000
Total Bus fare is: 2000
MRO: ['Minibus', 'Bus', 'Vehicle', 'object']
```

Q11. Create a Car class inheriting the Vehicle class, add a type attribute and print it using a display type method

In [59]:
```
class Vehicle:
    def __init__(self, name, mileage, capacity):
        self.name = name
        self.mileage = mileage
        self.capacity = capacity

    def fare(self):
        return self.capacity * 100

class Car(Vehicle):
    def __init__(self, name, mileage, capacity, type):
        super().__init__(name, mileage, capacity)   # access the parent class from inside the child
        self.type = type

    def display_type(self):
        print("Type of", self.name, "is:", self.type)

c1 = Car("Honda City", 18, 5, "Sedan")
print("Type attribute:", c1.type)
c1.display_type()
print("Total fare is:", c1.fare())
```

```
Type attribute: Sedan
Type of Honda City is: Sedan
Total fare is: 500
```

Q12. Method Overriding (Shape, Square and Circle)

`print(b)` calls `Shape.__str__()`, `b.fact()` calls `Shape.fact()`, `a.fact()` calls `Square.fact()` (overridden) and `b.area()` calls `Circle.area()`.

In [60]:
```
class Shape:
    def __str__(self):
        return "I am a two-dimensional shape."

    def fact(self):
        return "Shapes are two-dimensional."

class Square(Shape):
    def __init__(self, length):
        self.length = length

    def fact(self):                        # overrides Shape.fact
        return "Squares have each angle equal to 90 degrees."

class Circle(Shape):
    def __init__(self, radius):
        self.radius = radius

    def area(self):
        return 3.14159 * self.radius ** 2

a = Square(4)
b = Circle(7)

print(b)
print(b.fact())
print(a.fact())
print(b.area())
```

```
I am a two-dimensional shape.
Shapes are two-dimensional.
Squares have each angle equal to 90 degrees.
153.93791
```

Q13. Method Overriding (Vehicle, Car and Truck)

Vehicle: speed() max speed 150, change_gear() 5 gear, show() displays Vehicle. Car overrides speed (240) and change_gear (6). Truck overrides speed (200) and change_gear (8). `truck.show()` uses the parent method.

In [61]:
```
class Vehicle:
    def speed(self):
        print("Max speed 150")

    def change_gear(self):
        print("5 gear")
```

```
    def show(self):
        print("Display Vehicle")

class Car(Vehicle):
    def speed(self):
        print("Max speed 240")

    def change_gear(self):
        print("6 gear")

class Truck(Vehicle):
    def speed(self):
        print("Max speed 200")

    def change_gear(self):
        print("8 gear")

print("Truck")
truck = Truck()
truck.show()
truck.speed()
truck.change_gear()

print("\nCar")
car = Car()
car.show()
car.speed()
car.change_gear()

print("\nVehicle")
v = Vehicle()
v.show()
v.speed()
v.change_gear()
```

```
Truck
Display Vehicle
Max speed 200
8 gear

Car
Display Vehicle
Max speed 240
6 gear

Vehicle
Display Vehicle
Max speed 150
5 gear
```

Experiment 12

Student class with class variables and a class method

Create class with student_id and student_name as class variables

In [62]:
```
class Student:
    student_id = 101                # class variable
    student_name = "Aarav Sharma"   # class variable

    @classmethod
    def display(cls):
        try:
            print(f"{'student_id':<15}{cls.student_id}")
            print(f"{'student_name':<15}{cls.student_name}")
            print(f"{'student_class':<15}{cls.student_class}")
        except AttributeError as e:
            print(f"Attribute error caught: {e}")

# student_class is not added yet, so the exception handler runs
Student.display()
```

```
student_id     101
student_name   Aarav Sharma
Attribute error caught: type object 'Student' has no attribute 'student_class'
```

Add a new class variable student_class and display all attributes

In [63]:
```
Student.student_class = "SE A1"    # new class variable added from outside the class

Student.display()
```

```
student_id     101
student_name   Aarav Sharma
student_class  SE A1
```

Account and Customer classes (inheritance)

In [64]:
```
class Account:
    def __init__(self, number, balance):
        self.number = number
        self.balance = balance

    def deposit(self, amount):
        if amount <= 0:
            print("Deposit amount must be positive.")
        else:
            self.balance += amount
            print(f"Deposited {amount}. Balance is now {self.balance}.")

    def withdraw(self, amount):
        if amount > self.balance:
            print("Insufficient balance, withdrawal failed.")
        else:
            self.balance -= amount
            print(f"Withdrew {amount}. Balance is now {self.balance}.")


class Customer(Account):
    def __init__(self, number, balance, name, address, dob):
        super().__init__(number, balance)    # call the Account constructor
        self.name = name
        self.address = address
        self.dob = dob

    def display(self):
        print("Customer Details")
        print(f"{'Name':<17}{self.name}")
        print(f"{'Address':<17}{self.address}")
        print(f"{'DOB':<17}{self.dob}")
        print(f"{'Account Number':<17}{self.number}")
        print(f"{'Balance':<17}{self.balance}")

print("Classes Account and Customer defined successfully.")
```

```
Classes Account and Customer defined successfully.
```

Perform deposit and withdraw, then display customer details

In [65]:
```
c1 = Customer(1001, 5000, "Priya Patel", "Andheri, Mumbai", "12/05/2005")

c1.deposit(2000)
c1.withdraw(1500)
print()
c1.display()
```

```
Deposited 2000. Balance is now 7000.
Withdrew 1500. Balance is now 5500.

Customer Details
Name             Priya Patel
Address          Andheri, Mumbai
DOB              12/05/2005
Account Number   1001
Balance          5500
```

Shape, Circle and Rectangle classes (method overriding)

In [66]:
```
class Shape:
    def area(self, side):                # area of a square
        return side * side


class Circle(Shape):
    def area(self, radius):              # overrides Shape.area
        return 3.14 * radius * radius
```

```
class Rectangle(Shape):
    def area(self, length, breadth):      # overrides Shape.area
        return length * breadth

print("Classes Shape, Circle and Rectangle defined successfully.")
```

```
Classes Shape, Circle and Rectangle defined successfully.
```

Create objects and display the areas

In [67]:
```
side = 4
radius = 7
length = 10
breadth = 5

s = Shape()
c = Circle()
r = Rectangle()

print(f"Area of Square    : {s.area(side):.2f}")
print(f"Area of Circle    : {c.area(radius):.2f}")
print(f"Area of Rectangle : {r.area(length, breadth):.2f}")
```

```
Area of Square    : 16.00
Area of Circle    : 153.86
Area of Rectangle : 50.00
```

Employee and Manager classes (inheritance)

In [68]:
```
class Employee:
    def __init__(self, name, age, salary):
        self.name = name
        self.age = age
        self.salary = salary

    def display(self):
        print(f"Name   : {self.name}")
        print(f"Age    : {self.age}")
        print(f"Salary : {self.salary}")


class Manager(Employee):
    def __init__(self, name, age, salary, department):
        super().__init__(name, age, salary)   # call the Employee constructor
        self.department = department

    def displayManager(self):
        self.display()                        # reuse the parent method
        print(f"Department : {self.department}")

print("Classes Employee and Manager defined successfully.")
```

```
Classes Employee and Manager defined successfully.
```

Create a Manager object and test displayManager()

In [69]:
```
m1 = Manager("Rohan Mehta", 35, 85000, "Data Science")
m1.displayManager()
```

```
Name   : Rohan Mehta
Age    : 35
Salary : 85000
Department : Data Science
```

Experiment 13

In [70]:
```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
```

```
C:\Users\djsce.student\AppData\Local\Continuum\anaconda3\lib\site-packages\pandas\compat\_optional.py:138: UserWarning: Pandas requires version '2.7.0' or newer of 'numexpr' (version '2.6.8' currently installed).
  warnings.warn(msg, UserWarning)
```

In [71]:
```
titanic = pd.read_csv("titanic (1).csv")
titanic.head()
```

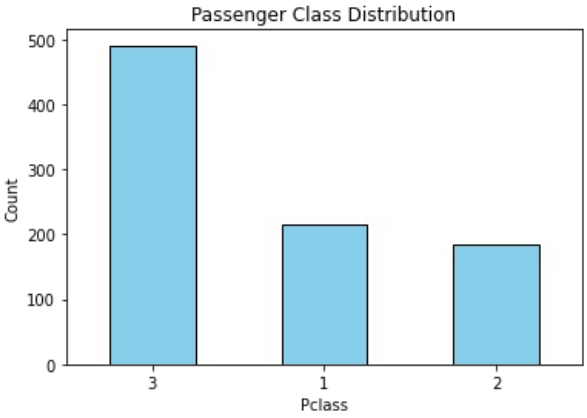
Out[71]:

	PassengerId	Survived	Pclass	Name	Sex	Age	SibSp	Parch	Ticket	Fare	Cabin	Embarked
0	1	0	3	Braund, Mr. Owen Harris	male	22.0	1	0	A/5 21171	7.2500	NaN	S
1	2	1	1	Cumings, Mrs. John Bradley (Florence Briggs Th...	female	38.0	1	0	PC 17599	71.2833	C85	C
2	3	1	3	Heikkinen, Miss. Laina	female	26.0	0	0	STON/O2. 3101282	7.9250	NaN	S
3	4	1	1	Futrelle, Mrs. Jacques Heath (Lily May Peel)	female	35.0	1	0	113803	53.1000	C123	S
4	5	0	3	Allen, Mr. William Henry	male	35.0	0	0	373450	8.0500	NaN	S

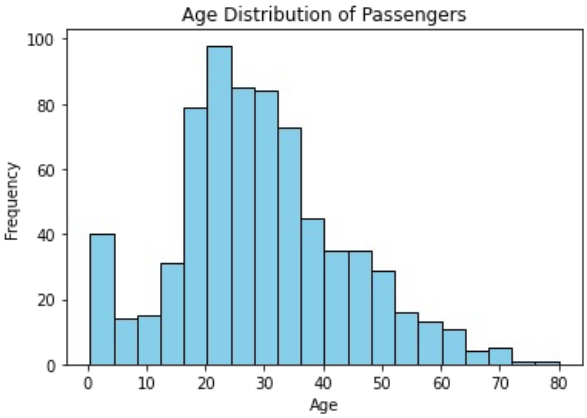
In [72]:
```
titanic.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 891 entries, 0 to 890
Data columns (total 12 columns):
 #   Column       Non-Null Count  Dtype  
---  ------       --------------  -----  
 0   PassengerId  891 non-null    int64  
 1   Survived     891 non-null    int64  
 2   Pclass       891 non-null    int64  
 3   Name         891 non-null    object 
 4   Sex          891 non-null    object 
 5   Age          714 non-null    float64
 6   SibSp        891 non-null    int64  
 7   Parch        891 non-null    int64  
 8   Ticket       891 non-null    object 
 9   Fare         891 non-null    float64
 10  Cabin        204 non-null    object 
 11  Embarked     889 non-null    object 
dtypes: float64(2), int64(5), object(5)
memory usage: 83.7+ KB
```

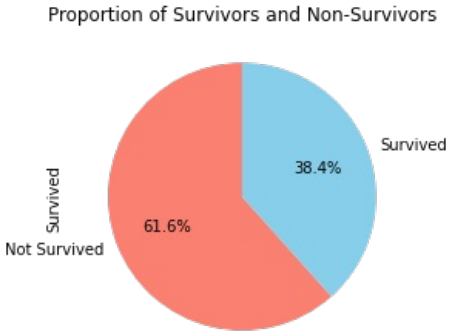
In [73]:
```
titanic['Pclass'].value_counts().plot(kind='bar', color='skyblue', edgecolor='black')
plt.title('Passenger Class Distribution')
plt.xlabel('Pclass')
plt.ylabel('Count')
plt.xticks(rotation=0)
plt.show()
```



In [74]:
```
titanic['Age'].plot(kind='hist', bins=20, color='skyblue', edgecolor='black')
plt.title('Age Distribution of Passengers')
plt.xlabel('Age')
plt.ylabel('Frequency')
plt.show()
```



In [75]:
```
titanic['Survived'].value_counts().plot(
    kind='pie',
    autopct='%1.1f%%',
    colors=['salmon', 'skyblue'],
    labels=['Not Survived', 'Survived'],
    startangle=90
)
plt.title('Proportion of Survivors and Non-Survivors')
plt.show()
```



In [76]:
```
iris = pd.read_csv("Iris.csv")
iris.head()
```

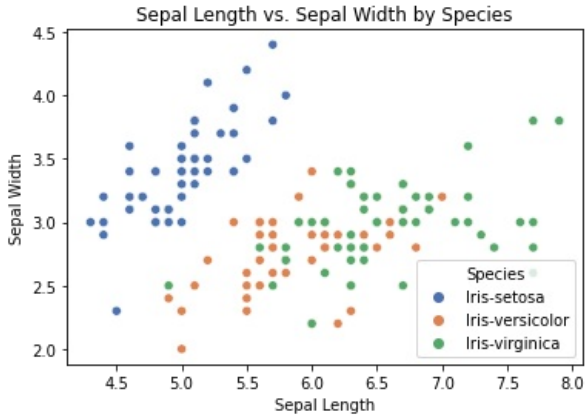
Out[76]:

	Id	SepalLengthCm	SepalWidthCm	PetalLengthCm	PetalWidthCm	Species
0	1	5.1	3.5	1.4	0.2	Iris-setosa
1	2	4.9	3.0	1.4	0.2	Iris-setosa
2	3	4.7	3.2	1.3	0.2	Iris-setosa
3	4	4.6	3.1	1.5	0.2	Iris-setosa
4	5	5.0	3.6	1.4	0.2	Iris-setosa

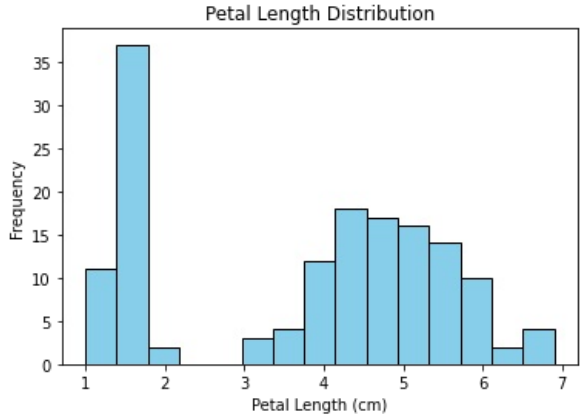
In [77]:
```
iris.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 150 entries, 0 to 149
Data columns (total 6 columns):
 #   Column         Non-Null Count  Dtype
---  ------         --------------  -----
 0   Id             150 non-null    int64
 1   SepalLengthCm  150 non-null    float64
 2   SepalWidthCm   150 non-null    float64
 3   PetalLengthCm  150 non-null    float64
 4   PetalWidthCm   150 non-null    float64
 5   Species        150 non-null    object
dtypes: float64(4), int64(1), object(1)
memory usage: 7.2+ KB
```

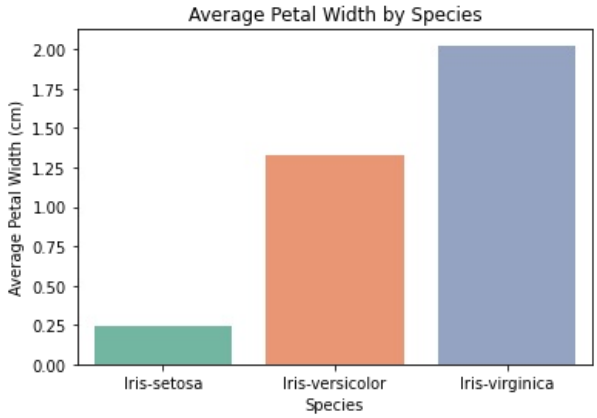
In [78]:
```
sns.scatterplot(data=iris, x='SepalLengthCm', y='SepalWidthCm', hue='Species', palette='deep')
plt.title('Sepal Length vs. Sepal Width by Species')
plt.xlabel('Sepal Length')
plt.ylabel('Sepal Width')
plt.show()
```



In [79]:
```
iris['PetalLengthCm'].plot(kind='hist', bins=15, color='skyblue', edgecolor='black')
plt.title('Petal Length Distribution')
plt.xlabel('Petal Length (cm)')
plt.ylabel('Frequency')
plt.show()
```



In [80]:
```
sns.barplot(data=iris, x='Species', y='PetalWidthCm', errorbar=None, palette='Set2')
plt.title('Average Petal Width by Species')
plt.xlabel('Species')
plt.ylabel('Average Petal Width (cm)')
plt.show()
```



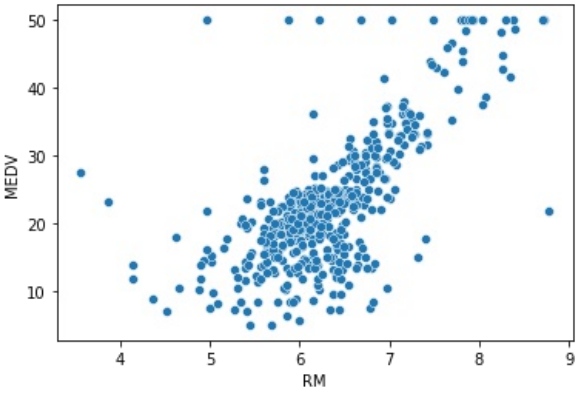
In [81]:
```
data_url = "http://lib.stat.cmu.edu/datasets/boston"
raw_df = pd.read_csv(data_url, sep=r"\s+", skiprows=22, header=None)
data = np.hstack([raw_df.values[::2, :], raw_df.values[1::2, :2]])
target = raw_df.values[1::2, 2]
columns = ['CRIM', 'ZN', 'INDUS', 'CHAS', 'NOX', 'RM', 'AGE', 'DIS', 'RAD', 'TAX', 'PTRATIO', 'B', 'LSTAT']
boston = pd.DataFrame(data, columns=columns)
boston['MEDV'] = target
```

In [82]:
```
boston.head()
```

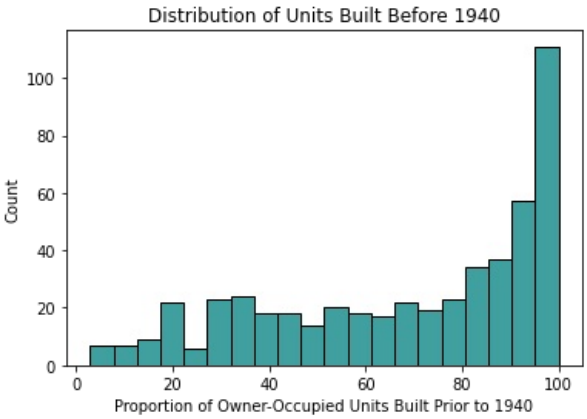
Out[82]:

	CRIM	ZN	INDUS	CHAS	NOX	RM	AGE	DIS	RAD	TAX	PTRATIO	B	LSTAT	MEDV
0	0.00632	18.0	2.31	0.0	0.538	6.575	65.2	4.0900	1.0	296.0	15.3	396.90	4.98	24.0
1	0.02731	0.0	7.07	0.0	0.469	6.421	78.9	4.9671	2.0	242.0	17.8	396.90	9.14	21.6
2	0.02729	0.0	7.07	0.0	0.469	7.185	61.1	4.9671	2.0	242.0	17.8	392.83	4.03	34.7
3	0.03237	0.0	2.18	0.0	0.458	6.998	45.8	6.0622	3.0	222.0	18.7	394.63	2.94	33.4
4	0.06905	0.0	2.18	0.0	0.458	7.147	54.2	6.0622	3.0	222.0	18.7	396.90	5.33	36.2

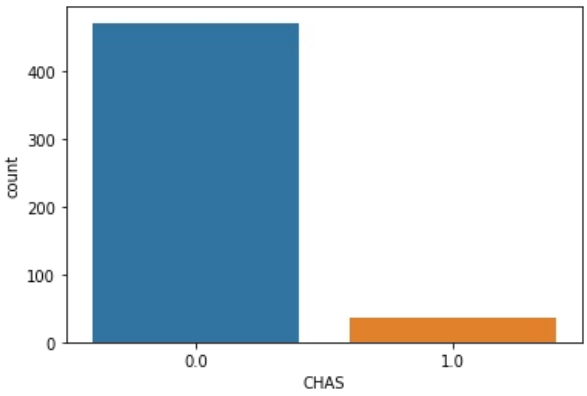
In [83]:
```
sns.scatterplot(data=boston, x='RM', y='MEDV')
plt.show()
```



In [84]:
```
sns.histplot(data=boston, x='AGE', bins=20, color='teal')
plt.title('Distribution of Units Built Before 1940')
plt.xlabel('Proportion of Owner-Occupied Units Built Prior to 1940')
plt.ylabel('Count')
plt.show()
```



In [85]:
```
sns.countplot(data=boston, x='CHAS')
plt.show()
```



In [86]:
```
wine = pd.read_csv("WineQT (1).csv")
wine.head()
```

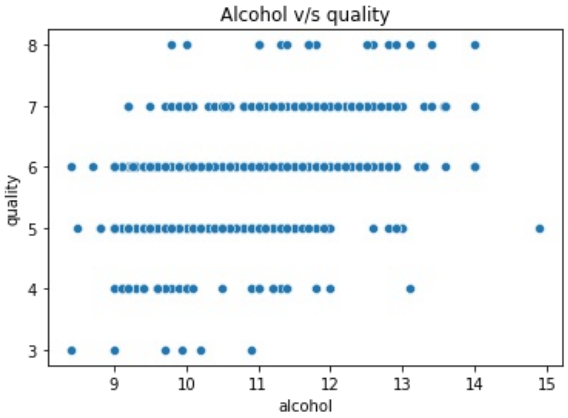
Out[86]:

	fixed acidity	volatile acidity	citric acid	residual sugar	chlorides	free sulfur dioxide	total sulfur dioxide	density	pH	sulphates	alcohol	quality	Id
0	7.4	0.70	0.00	1.9	0.076	11.0	34.0	0.9978	3.51	0.56	9.4	5	0
1	7.8	0.88	0.00	2.6	0.098	25.0	67.0	0.9968	3.20	0.68	9.8	5	1
2	7.8	0.76	0.04	2.3	0.092	15.0	54.0	0.9970	3.26	0.65	9.8	5	2
3	11.2	0.28	0.56	1.9	0.075	17.0	60.0	0.9980	3.16	0.58	9.8	6	3
4	7.4	0.70	0.00	1.9	0.076	11.0	34.0	0.9978	3.51	0.56	9.4	5	4

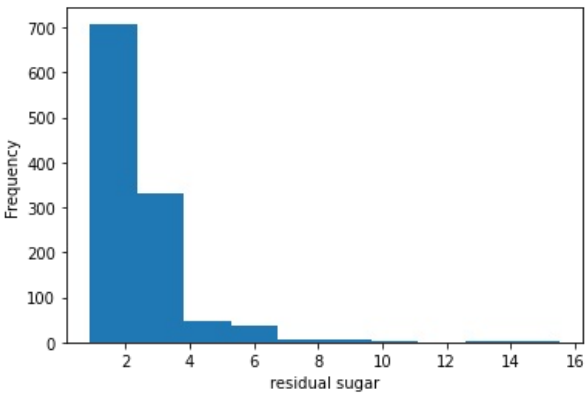
In [87]:
```
wine.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 1143 entries, 0 to 1142
Data columns (total 13 columns):
 #   Column                Non-Null Count  Dtype  
---  ------                --------------  -----  
 0   fixed acidity         1143 non-null   float64
 1   volatile acidity      1143 non-null   float64
 2   citric acid           1143 non-null   float64
 3   residual sugar        1143 non-null   float64
 4   chlorides             1143 non-null   float64
 5   free sulfur dioxide   1143 non-null   float64
 6   total sulfur dioxide  1143 non-null   float64
 7   density               1143 non-null   float64
 8   pH                    1143 non-null   float64
 9   sulphates             1143 non-null   float64
 10  alcohol               1143 non-null   float64
 11  quality               1143 non-null   int64  
 12  Id                    1143 non-null   int64  
dtypes: float64(11), int64(2)
memory usage: 116.2 KB
```

In [88]:
```
sns.scatterplot(data=wine, x='alcohol', y='quality')
plt.title("Alcohol v/s quality")
plt.show()
```

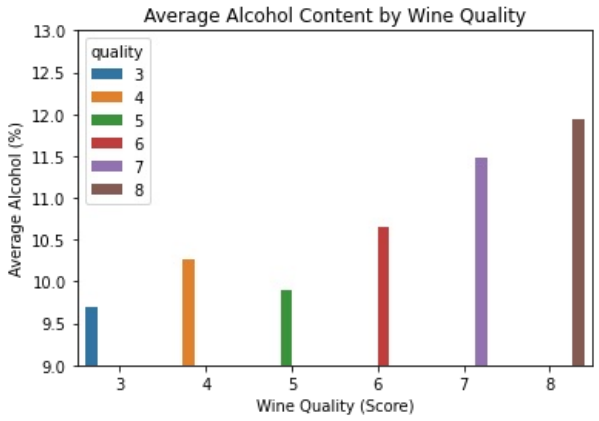


In [89]:
```
plt.hist(wine['residual sugar'])
plt.xlabel('residual sugar')
plt.ylabel('Frequency')
plt.show()
```



In [90]:
```
avg_alcohol = wine.groupby('quality')['alcohol'].mean().reset_index()
sns.barplot(x='quality', y='alcohol', data=avg_alcohol, hue='quality')
plt.title('Average Alcohol Content by Wine Quality')
plt.xlabel('Wine Quality (Score)')
```

```
plt.ylabel('Average Alcohol (%)')
plt.ylim(9, 13)
plt.show()
```



Experiment 14

Create database and connection

In [91]:
```
import sqlite3
from sqlite3 import Error

DB_NAME = "college.db"

try:
    conn = sqlite3.connect(DB_NAME)   # creates the file if it doesn't exist
    print(f"Connected to '{DB_NAME}' successfully.")
except Error as e:
    print(f"Connection error: {e}")
```

```
Connected to 'college.db' successfully.
```

Create STUDENT table

In [92]:
```
try:
    conn.execute("""
        CREATE TABLE IF NOT EXISTS STUDENT (
            Roll_No INTEGER PRIMARY KEY,
            Name    TEXT NOT NULL,
            Age     INTEGER CHECK (Age > 0),
            Branch  TEXT NOT NULL
        )
    """)
    conn.commit()
    print("Table STUDENT created successfully.")
except Error as e:
    print(f"Table creation error: {e}")
```

```
Table STUDENT created successfully.
```

Insert three records

In [93]:
```
students = [
    (101, "Aarav Sharma", 19, "AI & DS"),
    (102, "Priya Patel",  20, "Computer"),
    (103, "Rohan Mehta",  21, "IT"),
]

try:
    conn.executemany(
        "INSERT INTO STUDENT (Roll_No, Name, Age, Branch) VALUES (?, ?, ?, ?)",
        students
    )
    conn.commit()
    print("3 records inserted successfully.")
except sqlite3.IntegrityError as e:
    print(f"Insert failed (duplicate Roll_No): {e}")
except Error as e:
    print(f"Insert error: {e}")
```

```
3 records inserted successfully.
```

Update age based on Roll_No

In [94]:
```python
roll_no = 102
new_age = 22

try:
    cur = conn.execute(
        "UPDATE STUDENT SET Age = ? WHERE Roll_No = ?",
        (new_age, roll_no)
    )
    conn.commit()
    if cur.rowcount:
        print(f"Age of Roll_No {roll_no} updated to {new_age}.")
    else:
        print(f"No student found with Roll_No {roll_no}.")
except Error as e:
    print(f"Update error: {e}")
```

```
Age of Roll_No 102 updated to 22.
```

Delete a student record

In [95]:
```python
roll_no = 103

try:
    cur = conn.execute("DELETE FROM STUDENT WHERE Roll_No = ?", (roll_no,))
    conn.commit()
    if cur.rowcount:
        print(f"Student with Roll_No {roll_no} deleted.")
    else:
        print(f"No student found with Roll_No {roll_no}.")
except Error as e:
    print(f"Delete error: {e}")
```

```
Student with Roll_No 103 deleted.
```

Fetch and display all records

In [96]:
```python
try:
    cur = conn.execute("SELECT Roll_No, Name, Age, Branch FROM STUDENT ORDER BY Roll_No")
    rows = cur.fetchall()

    print(f"{'Roll_No':<10}{'Name':<15}{'Age':<6}{'Branch'}")
    print("-" * 45)
    if not rows:
        print("(no records)")
    for r in rows:
        print(f"{r[0]:<10}{r[1]:<15}{r[2]:<6}{r[3]}")
except Error as e:
    print(f"Fetch error: {e}")
```

```
Roll_No   Name           Age   Branch
---------------------------------------------
101       Aarav Sharma   19    AI & DS
102       Priya Patel    22    Computer
```

Exception handling demo (duplicate Roll_No)

In [97]:
```python
try:
    conn.execute(
        "INSERT INTO STUDENT (Roll_No, Name, Age, Branch) VALUES (?, ?, ?, ?)",
        (101, "Duplicate Entry", 20, "IT")
    )
    conn.commit()
except sqlite3.IntegrityError as e:
    print(f"IntegrityError caught: {e}")
except Error as e:
    print(f"Database error caught: {e}")
finally:
    conn.close()
    print("Connection closed.")
```

```
IntegrityError caught: UNIQUE constraint failed: STUDENT.Roll_No
Connection closed.
```